

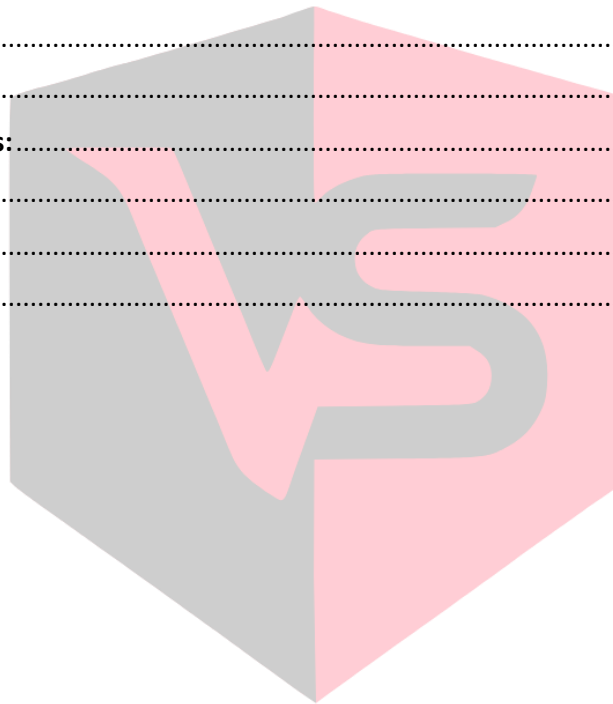
SMART CONTRACT AUDIT



VULSIGHT

Contents

Executive Summary:	3
Project Background:	3
Audit Scope:	3
Audit Summary:	3
Severity Definitions:	4
Technical Checks:	4
Code Quality and Documentation:	4
Audit Findings:	4
Critical vulnerabilities:	4
High vulnerabilities:	4
Medium vulnerabilities:	4
Low vulnerabilities:	5
Disclaimer:	5
Conclusion:	5



VULSIGHT

Executive Summary:

The client contacted our Firm to conduct a smart contract audit on their Solidity based smart contract. The audit was performed using both manual code analysis and automated security tools. The audit is performed on a best effort basis. No vulnerability was detected within the smart contract code during the course of this audit. All of the findings of the audit is detailed inside this report.

Audit conducted by: Waleed Ahmed



Project Background:

The smart contract powers the token of the client website with the token name "SCWALPHA". The smart contract code is written in solidity. The smart contract is deployed on the fantom network.

Audit Scope:

Token Name	SCWALPHA
Chain	Fantom
Language	Solidity
Audit Completion Date	21/10/22

Audit Summary:

The smart contract was found to be strongly secured. The contract doesn't have any "Owner" based functionality so it makes it fully decentralized.

Various tools such as Slither were used in the audit. Extensive manual code review was also done to ensure that any vulnerability if present, could be detected in the audit.

We found:

- **0 Critical risk vulnerabilities**
- **0 High risk vulnerabilities**
- **0 Medium risk vulnerabilities**
- **0 Low risk vulnerabilities**

Severity Definitions:

Risk Level	Description
Critical	Any kind of vulnerability that could lead to token or monetary loss
High	Any type of vulnerability that could disrupt the proper functioning of smart contract
Medium	Any type of vulnerability that could cause undesired actions but no serious disruption or monetary loss
Low	Any type of vulnerability that doesn't have any significant impact on the execution of the proper functioning of contract

Technical Checks:

Checks	Result
Solidity version not specified	Passed
Solidity version too old	Passed
Integer overflow/underflow	Passed
Function input parameters lack of check	Passed
Function input parameters check bypass	Passed
Insufficient randomness used	Passed
Fallback function misuse	Passed
Race Condition	Passed
Logical Vulnerabilities	Passed
Function visibility not explicitly declared	Passed
Use of deprecated keywords/functions	Passed
Out of Gas issue	Passed

Code Quality and Documentation:

The audit scope included one smart contract which was written in Solidity programming language. The code is well indented and clear in nature. The code was easy to understand.

All of the functions were clearly and properly commented. The commenting made the functionality of the functions easy and clear to understand.

Audit Findings:

Critical vulnerabilities:

0 critical vulnerabilities were found in the smart contract.

High vulnerabilities:

0 high vulnerabilities were found in the smart contract.

Medium vulnerabilities:

0 medium vulnerabilities were found in the smart contract.

Low vulnerabilities:

0 low vulnerabilities were found in the smart contract.

Disclaimer:

The smart contract was tested on a best-effort basis. The smart contract was analyzed with the best security practices known at the time of writing this report. Due to the fact that the total number of test cases is unlimited and the fact that new vulnerabilities in technologies are discovered every day, the audit report makes no guarantees on the security of the contract. It is recommended to not just rely on this audit report alone. A bug bounty program for this smart contract can be created to identify any vulnerabilities within the future.

Conclusion:

The smart contract was tested extensively both automatically and manually. No vulnerability of any kind was detected in it. It is recommended additional security measures be undertaken to ensure future security such that of a creation of a bug bounty program.

It is also recommended to use a proxy mechanism in the smart contract to ensure that the smart contract code can be modified in the future.

